

Batch normalization

In artificial neural networks, **batch normalization** (also known as **batch norm**) is a normalization technique used to make training faster and more stable by adjusting the inputs to each layer—re-centering them around zero and re-scaling them to a standard size. It was introduced by Sergey Ioffe and Christian Szegedy in 2015.^[1]

Experts still debate why batch normalization works so well. It was initially thought to tackle *internal covariate shift*, a problem where parameter initialization and changes in the distribution of the inputs of each layer affect the learning rate of the network.^[1] However, newer research suggests it does not fix this shift but instead smooths the objective function—a mathematical guide the network follows to improve—enhancing performance.^[2] In very deep networks, batch normalization can initially cause a severe gradient explosion—where updates to the network grow uncontrollably large—but this is managed with shortcuts called skip connections in residual networks.^[3] Another theory is that batch normalization adjusts data by handling its size and path separately, speeding up training.^[4]

Internal covariate shift

Each layer in a neural network has inputs that follow a specific distribution, which shifts during training due to two main factors: the random starting values of the network's settings (parameter initialization) and the natural variation in the input data. This shifting pattern affecting the inputs to the network's inner layers is called **internal covariate shift**. While a strict definition is not fully agreed upon, experiments show that it involves changes in the means and variances of these inputs during training.

Batch normalization was first developed to address internal covariate shift.^[1] During training, as the parameters of preceding layers adjust, the distribution of inputs to the current layer changes accordingly, such that the current layer needs to constantly readjust to new distributions. This issue is particularly severe in deep networks, because small changes in shallower hidden layers will be amplified as they propagate within the network, resulting in significant shift in deeper hidden layers. Batch normalization was proposed to reduced these unwanted shifts to speed up training and produce more reliable models.

Beyond possibly tackling internal covariate shift, batch normalization offers several additional advantages. It allows the network to use a higher learning rate—a setting that controls how quickly the network learns—without causing problems like vanishing or exploding gradients, where updates become too small or too large. It also appears to have a regularizing effect, improving the network's ability to generalize to new data, reducing the need for dropout, a technique used to prevent overfitting (when a model learns the training data too well and fails on new data). Additionally, networks using batch normalization are less sensitive to the choice of starting settings or learning rates, making them more robust and adaptable.

Procedures

Transformation

In a neural network, batch normalization is achieved through a normalization step that fixes the means and variances of each layer's inputs. Ideally, the normalization would be conducted over the entire training set, but to use this step jointly with stochastic optimization methods, it is impractical to use the global information. Thus, normalization is restrained to each mini-batch in the training process.

Let us use B to denote a mini-batch of size m of the entire training set. The empirical mean and variance of B could thus be denoted as

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \text{ and } \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

For a layer of the network with d -dimensional input, $\mathbf{x} = (x^{(1)}, \dots, x^{(d)})$, each dimension of its input is then normalized (i.e. re-centered and re-scaled) separately,

$$\hat{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_B^{(k)}}{\sqrt{(\sigma_B^{(k)})^2 + \epsilon}}, \text{ where } k \in [1, d] \text{ and } i \in [1, m]; \mu_B^{(k)} \text{ and } \sigma_B^{(k)} \text{ are the per-dimension mean and standard deviation, respectively.}$$

ϵ is added in the denominator for numerical stability and is an arbitrarily small positive constant. The resulting normalized activation $\hat{\mathbf{x}}^{(k)}$ have zero mean and unit variance, if ϵ is not taken into account. To restore the representation power of the network, a transformation step then follows as

$$y_i^{(k)} = \gamma^{(k)} \hat{x}_i^{(k)} + \beta^{(k)},$$

where the parameters $\gamma^{(k)}$ and $\beta^{(k)}$ are subsequently learned in the optimization process.

Formally, the operation that implements batch normalization is a transform $BN_{\gamma^{(k)}, \beta^{(k)}} : \mathbf{x}_{1\dots m}^{(k)} \rightarrow \mathbf{y}_{1\dots m}^{(k)}$ called the Batch Normalizing transform. The output of the BN transform $\mathbf{y}^{(k)} = BN_{\gamma^{(k)}, \beta^{(k)}}(\mathbf{x}^{(k)})$ is then passed to other network layers, while the normalized output $\hat{\mathbf{x}}_i^{(k)}$ remains internal to the current layer.

Backpropagation

The described BN transform is a differentiable operation, and the gradient of the loss l with respect to the different parameters can be computed directly with the chain rule.

Specifically, $\frac{\partial l}{\partial y_i^{(k)}}$ depends on the choice of activation function, and the gradient against other parameters could be expressed as a function of $\frac{\partial l}{\partial y_i^{(k)}}$:

$$\frac{\partial l}{\partial \hat{x}_i^{(k)}} = \frac{\partial l}{\partial y_i^{(k)}} \gamma^{(k)},$$

$$\frac{\partial l}{\partial \gamma^{(k)}} = \sum_{i=1}^m \frac{\partial l}{\partial y_i^{(k)}} \hat{x}_i^{(k)}, \quad \frac{\partial l}{\partial \beta^{(k)}} = \sum_{i=1}^m \frac{\partial l}{\partial y_i^{(k)}},$$

$$\frac{\partial l}{\partial \sigma_B^{(k)^2}} = \sum_{i=1}^m \frac{\partial l}{\partial y_i^{(k)}} (x_i^{(k)} - \mu_B^{(k)}) \left(-\frac{\gamma^{(k)}}{2} (\sigma_B^{(k)^2} + \epsilon)^{-3/2} \right),$$

$$\frac{\partial l}{\partial \mu_B^{(k)}} = \sum_{i=1}^m \frac{\partial l}{\partial y_i^{(k)}} \frac{-\gamma^{(k)}}{\sqrt{\sigma_B^{(k)^2} + \epsilon}} + \frac{\partial l}{\partial \sigma_B^{(k)^2}} \frac{1}{m} \sum_{i=1}^m (-2) \cdot (x_i^{(k)} - \mu_B^{(k)}),$$

$$\text{and } \frac{\partial l}{\partial x_i^{(k)}} = \frac{\partial l}{\partial \hat{x}_i^{(k)}} \frac{1}{\sqrt{\sigma_B^{(k)^2} + \epsilon}} + \frac{\partial l}{\partial \sigma_B^{(k)^2}} \frac{2(x_i^{(k)} - \mu_B^{(k)})}{m} + \frac{\partial l}{\partial \mu_B^{(k)}} \frac{1}{m}.$$

Inference

During the training stage, the normalization steps depend on the mini-batches to ensure efficient and reliable training. However, in the inference stage, this dependence is not useful any more. Instead, the normalization step in this stage is computed with the population statistics such that the output could depend on the input in a deterministic manner. The population mean, $E[x^{(k)}]$, and variance, $\text{Var}[x^{(k)}]$, are computed as:

$$E[x^{(k)}] = E_B[\mu_B^{(k)}], \text{ and } \text{Var}[x^{(k)}] = \frac{m}{m-1} E_B[(\sigma_B^{(k)})^2].$$

The population statistics thus is a complete representation of the mini-batches.

The BN transform in the inference step thus becomes

$$y^{(k)} = BN_{\gamma^{(k)}, \beta^{(k)}}^{\text{inf}}(x^{(k)}) = \gamma^{(k)} \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}] + \epsilon}} + \beta^{(k)},$$

where $y^{(k)}$ is passed on to next layers instead of $x^{(k)}$. Since the parameters are fixed in this transformation, the batch normalization procedure is essentially applying a linear transform to the activation function.

Theory

Although batch normalization has become popular due to its strong empirical performance, the working mechanism of the method is not yet well-understood. The explanation made in the original paper^[1] was that batch norm works by reducing internal covariate shift, but this has been challenged by more recent work. One experiment^[5] trained a VGG-16 network^[6] under 3 different training regimes: standard (no batch norm), batch norm, and batch norm with noise added to each layer during training. In the third model, the noise has non-zero mean and non-unit variance, i.e. it explicitly introduces covariate shift. Despite this, it showed similar accuracy to the second model, and both performed better than the first, suggesting that covariate shift is not the reason that batch norm improves performance.

Using batch normalization causes the items in a batch to no longer be iid, which can lead to difficulties in training due to lower quality gradient estimation.^[7]

Smoothness

One alternative explanation^[5] is that the improvement with batch normalization is instead due to producing a smoother parameter space and smoother gradients, as formalized by a smaller Lipschitz constant.

Consider two identical networks, one contains batch normalization layers and the other does not, the behaviors of these two networks are then compared. Denote the loss functions as $\hat{L}$ and L , respectively. Let the input to both networks be $\mathbf{x}$, and the output be $\mathbf{y}$, for which $\mathbf{y} = \mathbf{W}\mathbf{x}$, where $\mathbf{W}$ is the layer weights. For the second network, $\mathbf{y}$ additionally goes through a batch normalization layer. Denote the normalized activation as $\hat{\mathbf{y}}$, which has zero mean and unit variance. Let the transformed activation be $\mathbf{z} = \gamma\hat{\mathbf{y}} + \beta$, and suppose γ and β are constants. Finally, denote the standard deviation over a mini-batch $\hat{\mathbf{y}}_j \in \mathbb{R}^m$ as σ_j .

First, it can be shown that the gradient magnitude of a batch normalized network, $\|\nabla_{\mathbf{y}_i} \hat{L}\|$, is bounded, with the bound expressed as

$$\|\nabla_{\mathbf{y}_i} \hat{L}\|^2 \leq \frac{\gamma^2}{\sigma_j^2} \left(\|\nabla_{\mathbf{y}_i} L\|^2 - \frac{1}{m} \langle \mathbf{1}, \nabla_{\mathbf{y}_i} L \rangle^2 - \frac{1}{m} \langle \nabla_{\mathbf{y}_i} L, \hat{\mathbf{y}}_j \rangle^2 \right).$$

Since the gradient magnitude represents the Lipschitzness of the loss, this relationship indicates that a batch normalized network could achieve greater Lipschitzness comparatively. Notice that the bound gets tighter when the gradient $\nabla_{\mathbf{y}_i} \hat{L}$ correlates with the activation $\hat{\mathbf{y}}_i$, which is a common phenomena. The scaling of $\frac{\gamma^2}{\sigma_j^2}$ is also significant, since the variance is often large.

Secondly, the quadratic form of the loss Hessian with respect to activation in the gradient direction can be bounded as

$$(\nabla_{\mathbf{y}_j} \hat{L})^T \frac{\partial \hat{L}}{\partial \mathbf{y}_j \partial \mathbf{y}_j} (\nabla_{\mathbf{y}_j} \hat{L}) \leq \frac{\gamma^2}{\sigma^2} \left(\frac{\partial \hat{L}}{\partial \mathbf{y}_j} \right)^T \left(\frac{\partial L}{\partial \mathbf{y}_j \partial \mathbf{y}_j} \right) \left(\frac{\partial \hat{L}}{\partial \mathbf{y}_j} \right) - \frac{\gamma}{m\sigma^2} \langle \nabla_{\mathbf{y}_j} L, \hat{\mathbf{y}}_j \rangle \left\| \frac{\partial \hat{L}}{\partial \mathbf{y}_j} \right\|^2.$$

The scaling of $\frac{\gamma^2}{\sigma_j^2}$ indicates that the loss Hessian is resilient to the mini-batch variance, whereas the second term on the right hand side suggests that it becomes smoother when the Hessian and the inner product are non-negative. If the loss is locally convex, then the Hessian is positive semi-definite, while the inner product is positive if $\hat{g}_j$ is in the direction towards the minimum of the loss. It could thus be concluded from this inequality that the gradient generally becomes more predictive with the batch normalization layer.

It then follows to translate the bounds related to the loss with respect to the normalized activation to a bound on the loss with respect to the network weights:

$$\hat{g}_j \leq \frac{\gamma^2}{\sigma_j^2} (g_j^2 - m\mu_{g_j}^2 - \lambda^2 \langle \nabla_{y_j} L, \hat{y}_j \rangle^2), \quad \text{where} \quad g_j = \max_{\|x\| \leq \lambda} \|\nabla_w L\|^2 \quad \text{and} \\ \hat{g}_j = \max_{\|x\| \leq \lambda} \|\nabla_w \hat{L}\|^2.$$

In addition to the smoother landscape, it is further shown that batch normalization could result in a better initialization with the following inequality:

$$\|W_0 - \hat{W}^*\|^2 \leq \|W_0 - W^*\|^2 - \frac{1}{\|W^*\|^2} (\|W^*\|^2 - \langle W^*, W_0 \rangle)^2, \quad \text{where } W^* \text{ and } \hat{W}^* \text{ are the} \\ \text{local optimal weights for the two networks, respectively.}$$

Some scholars argue that the above analysis cannot fully capture the performance of batch normalization, because the proof only concerns the largest eigenvalue, or equivalently, one direction in the landscape at all points. It is suggested that the complete eigenspectrum needs to be taken into account to make a conclusive analysis.^{[8][5]}

Measure

Since it is hypothesized that batch normalization layers could reduce internal covariate shift, an experiment is set up to measure quantitatively how much covariate shift is reduced. First, the notion of internal covariate shift needs to be defined mathematically. Specifically, to quantify the adjustment that a layer's parameters make in response to updates in previous layers, the correlation between the gradients of the loss before and after all previous layers are updated is measured, since gradients could capture the shifts from the first-order training method. If the shift introduced by the changes in previous layers is small, then the correlation between the gradients would be close to 1.

The correlation between the gradients are computed for four models: a standard VGG network,^[6] a VGG network with batch normalization layers, a 25-layer deep linear network (DLN) trained with full-batch gradient descent, and a DLN network with batch normalization layers. Interestingly, it is shown that the standard VGG and DLN models both have higher correlations of gradients compared with their counterparts, indicating that the additional batch normalization layers are not reducing internal covariate shift.

Vanishing/exploding gradients

Even though batch norm was originally introduced to alleviate gradient vanishing or explosion problems, a deep batch norm network in fact *suffers from gradient explosion* at initialization time, no matter what it uses for nonlinearity. Thus, the optimization landscape is very far from smooth for a randomly initialized, deep batch norm network. More precisely, if the network has L layers, then the gradient of the first layer weights has norm $> c\lambda^L$ for some $\lambda > 1, c > 0$ depending only on the nonlinearity. For any fixed nonlinearity, λ decreases as the batch size increases. For example, for ReLU, λ decreases to $\pi/(\pi - 1) \approx 1.467$ as the batch size tends to infinity. Practically, this means deep batch norm networks are untrainable. This is only relieved by skip connections in the fashion of residual networks.^[9]

This gradient explosion on the surface contradicts the *smoothness* property explained in the previous section, but in fact they are consistent. The previous section studies the effect of inserting a single batch norm in a network, while the gradient explosion depends on stacking batch norms typical of modern deep neural networks.

Decoupling

Another possible reason for the success of batch normalization is that it decouples the length and direction of the weight vectors and thus facilitates better training.

By interpreting batch norm as a reparametrization of weight space, it can be shown that the length and the direction of the weights are separated and can thus be trained separately. For a particular neural network unit with input $\mathbf{x}$ and weight vector $\mathbf{w}$, denote its output as $\mathbf{f}(\mathbf{w}) = E_{\mathbf{x}}[\phi(\mathbf{x}^T \mathbf{w})]$, where ϕ is the activation function, and denote $\mathbf{S} = E[\mathbf{x}\mathbf{x}^T]$. Assume that $E[\mathbf{x}] = \mathbf{0}$, and that the spectrum of the matrix $\mathbf{S}$ is bounded as $0 < \mu = \lambda_{\min}(\mathbf{S}), L = \lambda_{\max}(\mathbf{S}) < \infty$, such that $\mathbf{S}$ is symmetric positive definite. Adding batch normalization to this unit thus results in

$$f_{BN}(\mathbf{w}, \gamma, \beta) = E_{\mathbf{x}}[\phi(BN(\mathbf{x}^T \mathbf{w}))] = E_{\mathbf{x}}\left[\phi\left(\gamma\left(\frac{\mathbf{x}^T \mathbf{w} - E_{\mathbf{x}}[\mathbf{x}^T \mathbf{w}]}{\text{var}_{\mathbf{x}}[\mathbf{x}^T \mathbf{w}]^{1/2}}\right) + \beta\right)\right], \text{ by definition.}$$

The variance term can be simplified such that $\text{var}_{\mathbf{x}}[\mathbf{x}^T \mathbf{w}] = \mathbf{w}^T \mathbf{S} \mathbf{w}$. Assume that $\mathbf{x}$ has zero mean and β can be omitted, then it follows that

$$f_{BN}(\mathbf{w}, \gamma) = E_{\mathbf{x}}\left[\phi\left(\gamma \frac{\mathbf{x}^T \mathbf{w}}{(\mathbf{w}^T \mathbf{S} \mathbf{w})^{1/2}}\right)\right], \text{ where } (\mathbf{w}^T \mathbf{S} \mathbf{w})^{1/2} \text{ is the induced norm of } \mathbf{S}, \|\mathbf{w}\|_s.$$

Hence, it could be concluded that $f_{BN}(\mathbf{w}, \gamma) = E_{\mathbf{x}}[\phi(\mathbf{x}^T \tilde{\mathbf{w}})]$, where $\tilde{\mathbf{w}} = \gamma \frac{\mathbf{w}}{\|\mathbf{w}\|_s}$, and γ and $\mathbf{w}$ account for its length and direction separately. This property could then be used to prove the faster convergence of problems with batch normalization.

Linear convergence

Least-square problem

With the reparametrization interpretation, it could then be proved that applying batch normalization to the ordinary least squares problem achieves a linear convergence rate in gradient descent, which is faster than the regular gradient descent with only sub-linear convergence.

Denote the objective of minimizing an ordinary least squares problem as

$$\min_{\tilde{w} \in \mathbb{R}^d} f_{OLS}(\tilde{w}) = \min_{\tilde{w} \in \mathbb{R}^d} (E_{x,y}[(y - x^T \tilde{w})^2]) = \min_{\tilde{w} \in \mathbb{R}^d} (2u^T \tilde{w} + \tilde{w}^T S \tilde{w}), \quad \text{where } u = E[-yx] \quad \text{and} \\ S = E[xx^T].$$

Since $\tilde{w} = \gamma \frac{w}{\|w\|_S}$, the objective thus becomes

$$\min_{w \in \mathbb{R}^d \setminus \{0\}, \gamma \in \mathbb{R}} f_{OLS}(w, \gamma) = \min_{w \in \mathbb{R}^d \setminus \{0\}, \gamma \in \mathbb{R}} \left(2\gamma \frac{u^T w}{\|w\|_S} + \gamma^2 \right), \quad \text{where } 0 \text{ is excluded to avoid } 0 \text{ in the denominator.}$$

Since the objective is convex with respect to γ , its optimal value could be calculated by setting the partial derivative of the objective against γ to 0. The objective could be further simplified to be

$$\min_{w \in \mathbb{R}^d \setminus \{0\}} \rho(w) = \min_{w \in \mathbb{R}^d \setminus \{0\}} \left(- \frac{w^T u u^T w}{w^T S w} \right).$$

Note that this objective is a form of the generalized Rayleigh quotient

$$\tilde{\rho}(w) = \frac{w^T B w}{w^T A w}, \quad \text{where } B \in \mathbb{R}^{d \times d} \text{ is a symmetric matrix and } A \in \mathbb{R}^{d \times d} \text{ is a symmetric positive definite matrix.}$$

It is proven that the gradient descent convergence rate of the generalized Rayleigh quotient is

$$\frac{\lambda_1 - \rho(w_{t+1})}{\rho(w_{t+1}) - \lambda_2} \leq \left(1 - \frac{\lambda_1 - \lambda_2}{\lambda_1 - \lambda_{\min}} \right)^{2t} \frac{\lambda_1 - \rho(w_t)}{\rho(w_t) - \lambda_2}, \quad \text{where } \lambda_1 \text{ is the largest eigenvalue of } B, \lambda_2 \text{ is the second largest eigenvalue of } B, \text{ and } \lambda_{\min} \text{ is the smallest eigenvalue of } B. \text{[10]}$$

In our case, $B = uu^T$ is a rank one matrix, and the convergence result can be simplified accordingly. Specifically, consider gradient descent steps of the form $w_{t+1} = w_t - \eta_t \nabla \rho(w_t)$ with step size

$$\eta_t = \frac{w_t^T S w_t}{2L|\rho(w_t)|}, \quad \text{and starting from } \rho(w_0) \neq 0, \text{ then}$$

$$\rho(w_t) - \rho(w^*) \leq \left(1 - \frac{\mu}{L} \right)^{2t} (\rho(w_0) - \rho(w^*)).$$

Learning halfspace problem

The problem of learning halfspaces refers to the training of the Perceptron, which is the simplest form of neural network. The optimization problem in this case is

$$\min_{\tilde{w} \in R^d} f_{LH}(\tilde{w}) = E_{y,x}[\phi(z^T \tilde{w})], \text{ where } z = -yx \text{ and } \phi \text{ is an arbitrary loss function.}$$

Suppose that ϕ is infinitely differentiable and has a bounded derivative. Assume that the objective function f_{LH} is ζ -smooth, and that a solution $\alpha^* = \operatorname{argmin}_{\alpha} \|\nabla f(\alpha w)\|^2$ exists and is bounded such that $-\infty < \alpha^* < \infty$. Also assume z is a multivariate normal random variable. With the Gaussian assumption, it can be shown that all critical points lie on the same line, for any choice of loss function ϕ . Specifically, the gradient of f_{LH} could be represented as

$$\nabla_{\tilde{w}} f_{LH}(\tilde{w}) = c_1(\tilde{w})u + c_2(\tilde{w})S\tilde{w}, \quad \text{where} \quad c_1(\tilde{w}) = E_z[\phi^{(1)}(z^T \tilde{w})] - E_z[\phi^{(2)}(z^T \tilde{w})](u^T \tilde{w}), \\ c_2(\tilde{w}) = E_z[\phi^{(2)}(z^T \tilde{w})], \text{ and } \phi^{(i)} \text{ is the } i\text{-th derivative of } \phi.$$

By setting the gradient to 0, it thus follows that the bounded critical points $\tilde{w}_*$ can be expressed as $\tilde{w}_* = g_* S^{-1}u$, where g_* depends on $\tilde{w}_*$ and ϕ . Combining this global property with length-direction decoupling, it could thus be proved that this optimization problem converges linearly.

First, a variation of gradient descent with batch normalization, Gradient Descent in Normalized Parameterization (GDNP), is designed for the objective function $\min_{w \in R^d \setminus \{0\}, \gamma \in R} f_{LH}(w, \gamma)$, such that the

direction and length of the weights are updated separately. Denote the stopping criterion of GDNP as

$$h(w_t, \gamma_t) = E_z[\phi'(z^T \tilde{w}_t)](u^T w_t) - E_z[\phi''(z^T \tilde{w}_t)](u^T w_t)^2.$$

Let the step size be

$$s_t = s(w_t, \gamma_t) = -\frac{\|w_t\|_S^3}{L g_t h(w_t, \gamma_t)}.$$

For each step, if $h(w_t, \gamma_t) \neq 0$, then update the direction as

$$w_{t+1} = w_t - s_t \nabla_w f(w_t, \gamma_t).$$

Then update the length according to

$\gamma_t = \mathbf{Bisection}(T_s, f, w_t)$, where $\mathbf{Bisection}()$ is the classical bisection algorithm, and T_s is the total iterations ran in the bisection step.

Denote the total number of iterations as T_d , then the final output of GDNP is

$$\tilde{w}_{T_d} = \gamma_{T_d} \frac{w_{T_d}}{\|w_{T_d}\|_S}.$$

The GDNP algorithm thus slightly modifies the batch normalization step for the ease of mathematical analysis.

It can be shown that in GDNP, the partial derivative of $\mathbf{f}_{LH}$ against the length component converges to zero at a linear rate, such that

$(\partial_\gamma \mathbf{f}_{LH}(\mathbf{w}_t, \mathbf{a}_t^{(T_s)})^2 \leq \frac{2^{-T_s} \zeta |\mathbf{b}_t^{(0)} - \mathbf{a}_t^{(0)}|}{\mu^2}$, where $\mathbf{a}_t^{(0)}$ and $\mathbf{b}_t^0$ are the two starting points of the bisection algorithm on the left and on the right, correspondingly.

Further, for each iteration, the norm of the gradient of $\mathbf{f}_{LH}$ with respect to $\mathbf{w}$ converges linearly, such that

$$\|\mathbf{w}_t\|_S^2 \|\nabla \mathbf{f}_{LH}(\mathbf{w}_t, \mathbf{g}_t)\|_{S^{-1}}^2 \leq \left(1 - \frac{\mu}{L}\right)^{2t} \Phi^2 \gamma_t^2 (\rho(\mathbf{w}_0) - \rho^*).$$

Combining these two inequalities, a bound could thus be obtained for the gradient with respect to $\tilde{\mathbf{w}}_{T_d}$:

$$\|\nabla_{\tilde{\mathbf{w}}} \mathbf{f}(\tilde{\mathbf{w}}_{T_d})\|^2 \leq \left(1 - \frac{\mu}{L}\right)^{2T_d} \Phi^2 (\rho(\mathbf{w}_0) - \rho^*) + \frac{2^{-T_s} \zeta |\mathbf{b}_t^{(0)} - \mathbf{a}_t^{(0)}|}{\mu^2},$$

such that the algorithm is guaranteed to converge linearly.

Although the proof stands on the assumption of Gaussian input, it is also shown in experiments that GDNP could accelerate optimization without this constraint.

Neural networks

Consider a multilayer perceptron (MLP) with one hidden layer and m hidden units with mapping from input $\mathbf{x} \in \mathbf{R}^d$ to a scalar output described as

$$\mathbf{F}_x(\tilde{\mathbf{W}}, \Theta) = \sum_{i=1}^m \theta_i \phi(\mathbf{x}^T \tilde{\mathbf{w}}^{(i)}),$$

where $\tilde{\mathbf{w}}^{(i)}$ and θ_i are the input and output weights of unit i correspondingly, and ϕ is the activation function and is assumed to be a tanh function.

The input and output weights could then be optimized with

$$\min_{\tilde{\mathbf{W}}, \Theta} (f_{NN}(\tilde{\mathbf{W}}, \Theta) = E_{y, \mathbf{x}} [l(-y \mathbf{F}_x(\tilde{\mathbf{W}}, \Theta))]),$$

where l is a loss function, $\tilde{\mathbf{W}} = \{\tilde{\mathbf{w}}^{(1)}, \dots, \tilde{\mathbf{w}}^{(m)}\}$, and $\Theta = \{\theta^{(1)}, \dots, \theta^{(m)}\}$.

Consider fixed Θ and optimizing only $\tilde{\mathbf{W}}$, it can be shown that the critical points of $\mathbf{f}_{NN}(\tilde{\mathbf{W}})$ of a particular hidden unit i , $\hat{\mathbf{w}}^{(i)}$, all align along one line depending on incoming information into the hidden layer, such that

$$\hat{\mathbf{w}}^{(i)} = \hat{c}^{(i)} \mathbf{S}^{-1} \mathbf{u},$$

where $\hat{c}^{(i)} \in \mathbf{R}$ is a scalar, $i = 1, \dots, m$.

This result could be proved by setting the gradient of $\mathbf{f}_{NN}$ to zero and solving the system of equations.

Apply the GDNP algorithm to this optimization problem by alternating optimization over the different hidden units. Specifically, for each hidden unit, run GDNP to find the optimal $\tilde{\mathbf{W}}$ and γ . With the same choice of stopping criterion and stepsize, it follows that

$$\|\nabla_{\tilde{w}^{(i)}} f(\tilde{w}_t^{(i)})\|_{S^{-1}}^2 \leq \left(1 - \frac{\mu}{L}\right)^{2t} C(\rho(w_0) - \rho^*) + \frac{2^{-T_s^{(i)}} \zeta |b_t^{(0)} - a_t^{(0)}|}{\mu^2}.$$

Since the parameters of each hidden unit converge linearly, the whole optimization problem has a linear rate of convergence.^[8]

References

1. Ioffe, Sergey; Szegedy, Christian (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". arXiv:1502.03167 (<https://arxiv.org/abs/1502.03167>) [cs.LG ([https://arxiv.org/archive/cs.LG](https://arxiv.org/archive/cs/LG))].
2. Santurkar, Shibani; Tsipras, Dimitris; Ilyas, Andrew; Madry, Aleksander (29 May 2018). "How Does Batch Normalization Help Optimization?". arXiv:1805.11604 (<https://arxiv.org/abs/1805.11604>) [stat.ML ([https://arxiv.org/archive/stat.ML](https://arxiv.org/archive/stat/ML))].
3. Yang, Greg; Pennington, Jeffrey; Rao, Vinay; Sohl-Dickstein, Jascha; Schoenholz, Samuel S. (2019). "A Mean Field Theory of Batch Normalization". arXiv:1902.08129 (<https://arxiv.org/abs/1902.08129>) [cs.NE ([https://arxiv.org/archive/cs.NE](https://arxiv.org/archive/cs/NE))].
4. Kohler, Jonas; Daneshmand, Hadi; Lucchi, Aurelien; Zhou, Ming; Neymeyr, Klaus; Hofmann, Thomas (27 May 2018). "Exponential convergence rates for Batch Normalization: The power of length-direction decoupling in non-convex optimization". arXiv:1805.10694 (<https://arxiv.org/abs/1805.10694>) [stat.ML ([https://arxiv.org/archive/stat.ML](https://arxiv.org/archive/stat/ML))].
5. Santurkar, Shibani; Tsipras, Dimitris; Ilyas, Andrew; Madry, Aleksander (29 May 2018). "How Does Batch Normalization Help Optimization?". arXiv:1805.11604 (<https://arxiv.org/abs/1805.11604>) [stat.ML ([https://arxiv.org/archive/stat.ML](https://arxiv.org/archive/stat/ML))].
6. Simonyan, Karen; Andrew, Zisserman (2014). "Very Deep Convolution Networks for Large Scale Image Recognition". arXiv:1409.1556 (<https://arxiv.org/abs/1409.1556>) [cs.CV ([https://arxiv.org/archive/cs.CV](https://arxiv.org/archive/cs/CV))].
7. Ba, J., Kiros, J.R., & Hinton, G.E. (2016). Layer Normalization. ArXiv, abs/1607.06450.
8. Kohler, Jonas; Daneshmand, Hadi; Lucchi, Aurelien; Zhou, Ming; Neymeyr, Klaus; Hofmann, Thomas (27 May 2018). "Exponential convergence rates for Batch Normalization: The power of length-direction decoupling in non-convex optimization". arXiv:1805.10694 (<https://arxiv.org/abs/1805.10694>) [stat.ML ([https://arxiv.org/archive/stat.ML](https://arxiv.org/archive/stat/ML))].
9. Yang, Greg; Pennington, Jeffrey; Rao, Vinay; Sohl-Dickstein, Jascha; Schoenholz, Samuel S. (2019). "A Mean Field Theory of Batch Normalization". arXiv:1902.08129 (<https://arxiv.org/abs/1902.08129>) [cs.NE ([https://arxiv.org/archive/cs.NE](https://arxiv.org/archive/cs/NE))].
10. Knyazev, Neymeyr (2003). "A geometric theory for preconditioned inverse iteration III: A short and sharp convergence estimate for generalized eigenvalue problems" (<https://doi.org/10.1016%2FS0024-3795%2801%2900461-X>). *Linear Algebra and Its Applications*. **358** (1–3): 95–114. doi:10.1016/S0024-3795(01)00461-X (<https://doi.org/10.1016%2FS0024-3795%2801%2900461-X>).

Further reading

- Ioffe, Sergey; Szegedy, Christian (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift", ICML'15: Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37, July 2015 Pages 448–456

- Simonyan, Karen; Zisserman, Andrew (2014). "Very Deep Convolutional Networks for Large-Scale Image Recognition". [arXiv:1409.1556](https://arxiv.org/abs/1409.1556) (<https://arxiv.org/abs/1409.1556>) [cs.CV (<https://arxiv.org/archive/cs.CV>)].
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Batch_normalization&oldid=1375137441"